B

FraudEngine-GapAnalysis

GOEMAN GLOBAL FINANCE · CONFIDENTIAL — FOUNDER · **JUNE 29, 2026**

Strategic guidance only — not legal, financial, tax, or investment advice. Securities, money-transmission, and corporate-formation decisions must be reviewed by licensed counsel and a CPA before you act. Sections flagged "⚖ see counsel" are where this matters most.

Contents

FraudEngine — Architecture Conformance & Gap Analysis

Subject: Does the Goeman Global Finance codebase conform to the target architecture in `FraudEngine.md`? **Date:** 2026-06-09 **Verdict: No — and by design.** `FraudEngine.md` is a production-scale, event-driven *target* architecture. The current repo is the TypeScript/Node prototype. Essentially none of the FraudEngine *platform* exists yet, and the prototype was never scoped to implement it. This document maps the gap precisely and proposes a phased path.

> **Update (2026-06-09): Stage 1 is now built.** The in-process fraud seam (§5 Stage 1) ships in `backend/src/services/fraudService.ts` + the append-only `fraud_decisions` table, screening the money path inside `transferService`. This **closes §4** (there is now a transaction-time fraud check).

> **Update (2026-06-13): Stages 2–4 are now built as a standalone add-on (`fraud-engine/`).** The full platform — event backbone + schema registry, per-user feature store + enrichment, a `rules-v1` ensemble plus a `seq-v0` sequence model (Transformer stand-in), a model **registry + serving** layer with **config-driven routing and shadow/canary**, an append-only decision topic, an analyst **case queue**, an **async remediation** loop that calls back into Goeman to **freeze/flag**, and a **label → retrain** loop — runs in `fraud-engine/` as a separate Node/TS service that imports **nothing** from `backend/`. Goeman uses it over HTTP via a hybrid integration: an in-Goeman triage decides blocking-vs-fire-and-forget; the remote score is advisory, the local deterministic gate + account-freeze are authoritative. Each layer is a prototype-scale stand-in behind an interface that maps 1:1 to the north-star tech (Kafka/Flink/Triton/MLflow/lakehouse), which remain the production graduation — see `fraud-engine/README.md`. The component table below describes the *production platform* gap; the **add-on implements the architecture, not the production infrastructure**.

> Scope note: this is an analysis deliverable. No runtime code was changed. Findings reflect the repo at the commit this file was added.

---

1 1. Executive summary

`FraudEngine.md` describes a **real-time fraud intelligence platform**: every product streams immutable events to a unified Kafka backbone; a fraud-owned Flink layer enriches via a feature store, routes each event to a versioned Transformer model (shadow/canary/prod),

emits a scored decision back to the stream, and continuously retrains from a lakehouse — all at sub-100 ms, with full audit and explainability.

The Goeman Global Finance prototype implements **none of that infrastructure** and, more importantly, has **no transaction-time fraud screening at all**. The money path (`transferService`, `smartchatService`, `ledgerService`) gates a transfer only on balance + idempotency (plus the compliance module for marketplace assets). No event is emitted, scored, or acted on for fraud.

This is consistent with the program's stated boundaries: `CLAUDE.md` lists Temporal/Conductor and equivalent production infra as **out of scope for current phases**, and `docs/GOEMAN-PLAN.md` (phases 0–16) contains **no fraud phase**. `FraudEngine.md` is best read as a **v2 / production north-star**, in the same category as the Go reimplementation and Temporal orchestration — directional, not a build spec the prototype was meant to satisfy.

---

2 2. Component-by-component conformance

FraudEngine component	Target tech	Status in repo	Evidence
Unified event backbone	Kafka/Redpanda + Schema Registry (Avro/Protobuf)	❌ Absent	No streaming deps in `backend/package.json`; no producer/consumer code
Fraud listening/processing	Apache Flink, per-user-partitioned stateful jobs	❌ Absent	No Flink, no consumer groups, no stream-state code
Real-time feature store	Tecton / Databricks online store (Redis/Cassandra)	❌ Absent	No feature store; no Redis/Cassandra
Dynamic model router	Flink ProcessFunction + config service (etcd/Consul)	❌ Absent	No routing layer; no model-selection config
Transformer model serving	Triton / vLLM / SageMaker, multi-version	❌ Absent	No model serving anywhere in the stack
Model registry + experiments	MLflow / Databricks Model Registry	❌ Absent	No registry; no versioned model artifacts
Shadow / canary / A-B testing	Parallel jobs, hash-based traffic split, instant promote/rollback	❌ Absent	No traffic-splitting or shadow-eval mechanism
Training lakehouse + retraining	Databricks Delta / Iceberg + Airflow	❌ Absent	No lakehouse; no training pipeline
Decision → stream → action	Output Kafka topic consumed by orchestrator/products	❌ Absent	No decision topic; no real-time action loop
Transaction-time fraud scoring	(implied by the whole design)	🟡 **Stage 1 built**	`fraudService.screenTransfer` screens the money path in `transferService`; deterministic `rules-v0` scorer (velocity/spike/new-payee/large-absolute) →

FraudEngine component	Target tech	Status in repo	Evidence
			allow/flag/challenge/block. Not yet a Transformer, not streaming
Compliance & audit	Per-decision model version + SHAP → audit topic	🟡 Partial analog	Append-only `audit_logs` / `mcp_audit_logs` (auth/ledger/MCP). Stage 1 adds append-only `fraud_decisions` with `model_version` + reasons (the "audit topic" shape) — reasons are rule codes, not yet SHAP

Legend: ✅ conforms · 🟡 partial / analog only · ❌ absent

---

3 3. What exists that is *adjacent* (and why it is not the fraud engine)

Two Phase 5A services rhyme with the design but are **onboarding-scoped**, synchronous, and rule-based — not ongoing transaction fraud:

- `backend/src/services/signalService.ts` — deterministic, rule-based scoring of *onboarding* signals (disposable email, IP, device fingerprint, rapid-completion) into PII-free sub-scores. PII minimization at the boundary. No streaming, no model serving.
- `backend/src/services/riskOrchestratorService.ts` — an *advisory* model (`utils/orchestratorModel.assessRisk`) whose output is overridden by deterministic guardrails in `finalizeDecision` (the single authority for a tier grant). Embodies "agents decide; deterministic code executes; humans gate."

Two design *threads* are genuinely shared and worth preserving into any future build:

1. **Advisory model + deterministic enforcement.** FraudEngine's "risk score + explanation → decision → action" maps cleanly onto the existing `assessRisk → finalizeDecision` split. A fraud engine should reuse this invariant: the Transformer is advisory; deterministic policy and human gates decide.
2. **Append-only audit as the "audit topic" analog.** `auditService` + append-only triggers on `audit_logs` / `mcp_audit_logs` are the closest existing thing to the doc's immutable audit stream — but they record auth/ledger/MCP events, not fraud decisions with model version + confidence + explanation.

The **compliance module** (`complianceService.ts`: tier/jurisdiction/holder-cap gating, `COMPLIANCE_BLOCKED`) is a *policy* gate on marketplace transfers, not a *fraud/anomaly* detector. It is rules, not risk scoring, and does not cover ordinary cash transfers.

Net: the prototype has onboarding risk scoring and policy/compliance gating, but **no real-time, transaction-level fraud detection** and **none** of the streaming/ML platform FraudEngine specifies.

---

4 4. The single most material gap

Independent of the heavy platform, the prototype has **no transaction-time fraud check on the money path**. `transferService.executeTransfer` gates only on:

- idempotency (inside the DB transaction, no TOCTOU), and
- balance sufficiency (inside the same transaction).

There is no velocity check, no anomaly score, no device/session correlation, no "challenge/flag/block" outcome distinct from a hard balance failure. This is the one gap that is both **high-value** and **achievable at prototype scale** without any new infrastructure — see Stage 1 below.

---

5 5. Phased roadmap (prototype → FraudEngine target)

Framed so each stage is independently shippable and each preserves the invariants above. Stages 2+ are explicitly v2/production and require the locked-architecture review that Kafka/Flink/model-serving demand.

Stage 0 — Status quo (today)

Onboarding-only `signalService` + `riskOrchestratorService`; compliance gating on assets; append-only audit. No transaction fraud.

Stage 1 — In-process fraud seam *(prototype-scale, no new infra)* — ✅ BUILT (2026-06-09)

- ✅ Normalized `TransferRiskEvent` emitted from the money path inside `transferService.transfer` (channel-tagged: `api` / `smartchat` / `mcp`) — an in-process event abstraction that *later* maps to a Kafka topic 1:1.

- ✅ Deterministic `fraudService.scoreTransferFeatures` (velocity, amount-vs-history spike, new-payee, large-absolute) returning `score + reasons + action ∈ {allow, flag, challenge, block}`. Pure and unit-tested.
- ✅ Enforced via the advisory+deterministic split (score is advisory; the thresholds in `fraudService` are the only thing that blocks). `block` → `FRAUD_BLOCKED`; `FRAUD_ENGINE_ENFORCE=false` gives shadow mode. The existing `>$500 → MFA` SmartChat gate remains the live "challenge".
- ✅ Each decision written to the append-only `fraud_decisions` table with score, reasons, and `model_version='rules-v0'` (+ mirrored to `audit_logs`) — the forward-compatible "audit topic" shape. A `fraud_decision_total{action}` prom counter is incremented.
- ✅ Only **funded** transfers are screened (an unfunded attempt stays `INSUFFICIENT_FUNDS`); the in-transaction balance check remains authoritative for TOCTOU.
- **Closes §4.** Tests: `backend/test/fraud.test.ts` (10) — pure scorer, allow/block/shadow on the money path, append-only enforcement, unfunded skip. Full suite 141 pass / 3 todo.

Stage 2 — Stream backbone — ✅ BUILT as `fraud-engine/` (2026-06-13)

`bus/eventBus.ts` (`EventBus` interface + `InProcessEventBus`, consumer groups, topics) is the broker analog; `bus/schemaRegistry.ts` validates versioned event schemas on ingest. `POST /v1/events` ingests; scoring runs in a decision pipeline; decisions publish to a `decisions` topic consumed by the remediation service. **Production graduation:** swap `InProcessEventBus` for a Kafka/Redpanda client behind the same interface; add cross-process exactly-once.

Stage 3 — Stateful enrichment + feature store — ✅ BUILT

`features/featureStore.ts` (`SqliteFeatureStore`) holds per-user sequence state (velocity window, trailing max, distinct payees, recent-amount sequence, last geo/device); `features/enrichment.ts` joins the snapshot before scoring. **Production graduation:** replace the SQLite store with a Tecton/Redis online store + Flink-partitioned state behind the `FeatureStore` interface.

Stage 4 — Model serving + registry + shadow/canary — ✅ BUILT

`models/registry.ts` (versions + `prod|shadow|canary|retired`), `models/serving.ts` (`ModelServer` + `ServingBackend` seam), `router/router.ts` (config-driven prod ensemble + shadow + hash-bucketed canary, live promotion), and `learning/retrain.ts` (labels → drift → registered shadow candidate). `rules-v1` is the prod floor; `seq-v0` is the

served sequence model (Transformer stand-in). **Production graduation:** register a real Triton/vLLM endpoint as a `ServingBackend`, back the registry with MLflow, and feed retraining from a real lakehouse. This is the full FraudEngine.md target architecture, with the heavy infra as the remaining swap.

---

6 6. Recommendations

1. **Reclassify `FraudEngine.md` as a v2/production north-star** in the docs, alongside Temporal/Conductor and the Go reimplementation, so it is not mistaken for current-phase scope. (Add a one-line banner at its top and a pointer from `docs/GOEMAN-PLAN.md`'s out-of-scope section.)
2. **If any fraud capability is wanted now, do Stage 1 only.** It is the high-value, achievable slice and it deliberately shapes its event/decision/audit contract to be forward-compatible with the Kafka/Flink target — so Stage 1 is throwaway-free.
3. **Do not introduce Kafka/Flink/model-serving unprompted.** Stages 2–4 are a multi-quarter platform effort and a locked-architecture decision; they need explicit scoping, a chain decision, and infra ownership before any code.
4. **Preserve the two shared invariants** in whatever gets built: advisory model + deterministic enforcement, and immutable per-decision audit with model version + explanation.